SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM049

Name: Gouri Rajendra Kudalkar

Division: A

Project Title: Graph Search System Using BFS and DFS

GitHub LINK : https://github.com/kudalkargouri-crypto/IAI_SLE-2.git

https://github.com/kudalkargouri-crypto/SLE3.git

Part A — Architecture Decision Record (ADR)

ADR Title: Modular Architecture for the BFS and DFS Graph Search System

Status: Accepted

Date: 11 October 2026

1. Context

The project is a Graph Search System that uses Breadth-First Search (BFS) and Depth-First Search (DFS). The graph contains 800 nodes, with node 0 as the starting node and node 799 as the goal node. The system searches for the goal and records execution time, the number of nodes expanded and whether the goal was found. BFS explores nodes level by level using a queue, while DFS explores deeper paths using a stack. I needed to compare the performance of both algorithms and organize the system into understandable components.

2. Decision

I decided to separate graph representation from the search algorithms so that BFS and DFS could be implemented and compared independently. I used a queue for BFS and a stack for DFS. I also included a visited structure and performance measurement to support the search and analysis.

3. Alternatives Considered

- **Alternative 1: Use only BFS.** This would allow level-by-level graph exploration but would not provide a direct comparison with DFS.
- **Alternative 2: Use only DFS.** This would explore deeper paths first but would not allow comparison with BFS.
- **Alternative 3:** Use a modular architecture that separates graph representation, search algorithms, visited-node management and output.

I selected the third alternative because it provides better organization, easier testing and simpler maintenance.

SLE-2: Empirical Performance Analysis

In SLE-2, I compared BFS and DFS using a graph of 800 nodes, with node 0 as the start and node 799 as the goal. I measured execution time and nodes expanded using Python profiling tools. Both algorithms found the goal and expanded 800 nodes. These results helped me justify my architectural decision.

Performance Comparison of BFS and DFS (SLE-2)

Metric	**BFS**	**DFS**
Graph size	800 nodes	800 nodes
Start node	0	0
Goal node	799	799
Best time	175.03617 ms	186.90420 ms
Average time	191.69571 ms	203.76627 ms
Worst time	217.95790 ms	232.18527 ms
Nodes expanded	800	800
Goal found	Yes	Yes

The reported average execution time of BFS was lower than that of DFS in my SLE-2 results. However, execution time can vary between runs. Both algorithms successfully found the goal node. This experiment helped me understand why performance measurement is important when comparing search algorithms.

SLE-3: Architectural Design Using the Full C4 Model

In SLE-3, I designed the Graph Search System using the C4 model: Context, Container, Component and Code. The architecture includes Input, Graph Management, Search Engine, Visited/Memory Management and Output modules. This design shows how the system is organized and how its components work together.

C4 Architecture Levels Used in SLE-3

C4 Level	What It Explains in My Project
Level 1 – Context	Shows how the user interacts with the graph search system.
Level 2 – Container	Shows the main modules, such as input, graph management, search and output.
Level 3 – Component	Shows the internal responsibilities of the search system and its components.
Level 4 – Code	Shows how the design is represented at the code level.

The SLE-3 architecture separates graph management from the search algorithms. BFS uses a queue to manage the search frontier, whereas DFS uses a stack. The visited-node structure helps track explored nodes, and the output module presents the search results. This separation makes the system easier to understand, test and modify. It also supports the modular architecture decision documented in SLE-4.

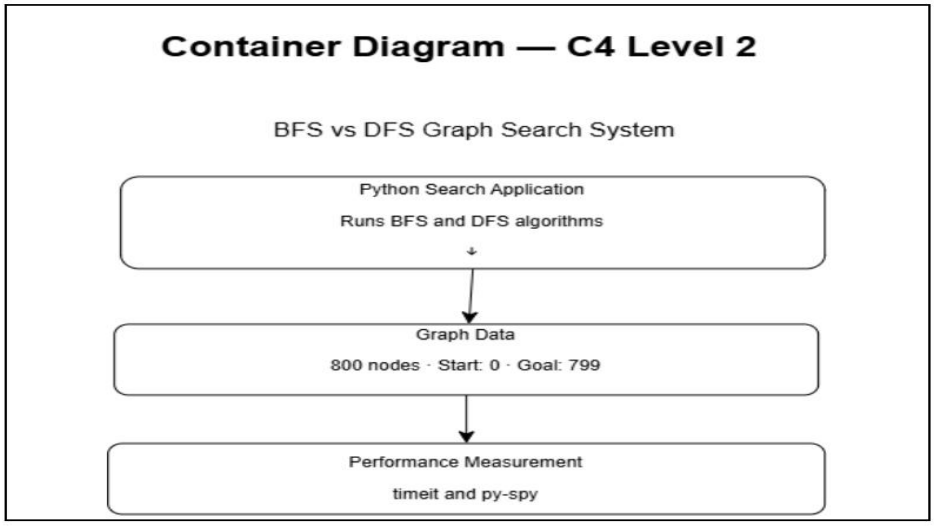


Figure 2: Container Diagram (C4 Level 2) of the BFS and DFS Graph Search System.

4. Rationale / Justification

I selected a modular architecture because BFS and DFS can use the same graph while following different search strategies. BFS uses a queue, whereas DFS uses a stack. In SLE-2, BFS recorded an average time of 191.69571 ms, while DFS recorded 203.76627 ms. Both algorithms found the goal and expanded 800 nodes. BFS was faster in this sample, but execution time can vary. The C4 design from SLE-3 supports the separation of graph management, search, visited-node tracking and output. This architecture improves readability, testing and maintenance.

Part B: Short Summary of My SLE Journey

In SLE-1, I worked on my AI project and its implementation. In SLE-2, I compared BFS and DFS through empirical performance analysis. I recorded execution time and nodes expanded to understand their performance. In SLE-3, I designed the system architecture using the C4 model, including Context, Container, Component and Code levels. In SLE-4, I documented my architectural decision and justified it using the results and design knowledge gained from SLE-2 and SLE-3.

5. Consequences

Positive consequences:

- BFS and DFS can be implemented and compared independently.
- The graph representation can be reused by both algorithms.
- The visited structure helps prevent repeated exploration of nodes.
- Performance measurements help compare the algorithms using experimental evidence.
- The separated components make the system easier to understand and maintain.

Negative consequences / Trade-offs:

- Maintaining separate search algorithms requires additional code.
- Performance measurements may vary because of system and runtime conditions.
- The results from one graph may not apply to every graph.
- A larger graph may require more processing time and memory.

Part C: AI Contribution Note

- **AI tool used:** ChatGPT
- **Contribution of AI:** AI helped me understand the ADR structure, BFS and DFS concepts, performance analysis and C4 architecture. It also helped me organize and improve the explanations.
- **My contribution:** I worked on the project, performed the performance analysis, reviewed the results, developed the architecture diagrams and checked the final documentation.

6. References

- SLE-2: Profiling Report — BFS vs DFS.
- SLE-3: Architectural Design (Full C4 Model).
- GitHub: SLE-2 and SLE-3 project repositories.

7.Conclusion:

The SLE-2 and SLE-3 activities helped in analyzing the performance and designing the architecture of the Graph Search System using BFS and DFS. In SLE-2, both algorithms were tested on a graph containing 800 nodes, with node 0 as the starting point and node 799 as the goal. BFS achieved an average execution time of 191.69571 ms, while DFS took 203.76627 ms. Both algorithms expanded 800 nodes and successfully found the goal. In this experiment, BFS performed faster than DFS.

In SLE-3, the C4 model was used to represent the system at the Context, Container, Component, and Code levels. The system was organized into separate modules for graph input and management, search algorithms, performance measurement, and result output. This modular architecture makes the system easier to understand, test, maintain, and modify.

Based on the performance results and architectural analysis, the decision to use a modular design was accepted. It supports a clear comparison between BFS and DFS while allowing each algorithm to be maintained independently. Overall, this work improved the understanding of algorithm analysis, software architecture, performance evaluation, and Architecture Decision Records (ADR). It also demonstrated how measured results and architectural design can support informed software development decisions.